



AutoGrader+

Your repository, marked like a code review

Multi-agent grading + a full-stack learning platform

Live: autograder-plus.onrender.com

Code: github.com/roshanraj9136/auto-grader

CSL100 group project

Push your project. Get it marked like a code review.

AutoGrader+ reads your whole GitHub repo, from the frontend to the Dockerfile, and gives back a score, the exact files to fix, and what to learn next. Usually in under a minute.

[Create a student account](#) [Sign in](#)

Just looking? Sign in with: `student@autograder.local` and `student123`.

76 /100 **Grade B**

Category	Score
Code quality	81
Architecture	79
Security	62
Testing	84
Docker	84

Fix this first
Hash passwords before saving them in `src/routes/auth.js`

One course, two views
Students see what to do next. Instructors see who needs help.

Class overview

8 students, 4 assignments. Start with the items marked in red.

[+ New assignment](#) [Gradebook](#)

Students active	Submissions	Class average	Averaging below 50	Shared repositories
7 of 8	10 / 10 graded	59.4 best try	2	2

Needs your attention (4 items)

- 2 repositories submitted by more than one student
dockerexamples/example-voting-app for Lab 1- Responsive portfolio site; docker/getting-started-app for Lab 1- Responsive portfolio site. Group work, or copying? [Show them](#)
- 2 students averaging below 50
Kolar Singh, Vignesh Nar [See students](#)
- 1 student hasn't submitted anything
Rohan Gupta [See students](#)
- Docker & DevOps is the class's weakest area
It averages 4.6 out of 10. A good topic for the next lab or lecture.

Grade distribution (best try per student and assignment)

Grade	Count
A	0
B	2
C	3
D	1
F	0

Assignments

Assignment	Due	Submitted	Average	Top	Attempts
Lab 1 - Responsive portfolio site	Oct 16, 2026	6 of 8	57	68.7	6
Lab 2 - REST API + PostgreSQL	Oct 22, 2026	4 of 8	63	74.2	4
Lab 3 - Containers, networks & load balancing	Oct 29, 2026	0 of 8	-	-	0
Project - Full-stack app with authentication	Nov 12, 2026	0 of 8	-	-	0

Why we built it

Grading full-stack projects by hand

- **Slow.** Every project has a frontend, an API, a database and a Dockerfile to read.
- **Inconsistent.** Two graders give the same project different marks.
- **Late feedback.** Students hear what was wrong after the deadline, when it no longer helps.
- **No overview.** Instructors can't easily see who is struggling or who copied.

What we wanted instead

- **Feedback in about a minute,** with the file and the fix.
- **Reproducible scores:** the total is computed in code from published weights.
- **Students improve and resubmit;** their best attempt counts.
- **Instructors see the class:** who needs help, the distribution, possible copying.

What AutoGrader+ is

Grading engine

5 specialist reviewers run in parallel
1 judge calibrates and explains
score, priorities, learning path

Student view

assignments with visible rubrics
live progress, detailed feedback
dashboard, XP, labs

Instructor view

what needs attention, gradebook
assignment analytics, copy detection
grade adjustment, re-runs

Five reviewers, one area each

Code quality

readable, well-structured
code

Architecture

layers, coupling, API and DB
design

Security

OWASP Top 10, secrets

Testing

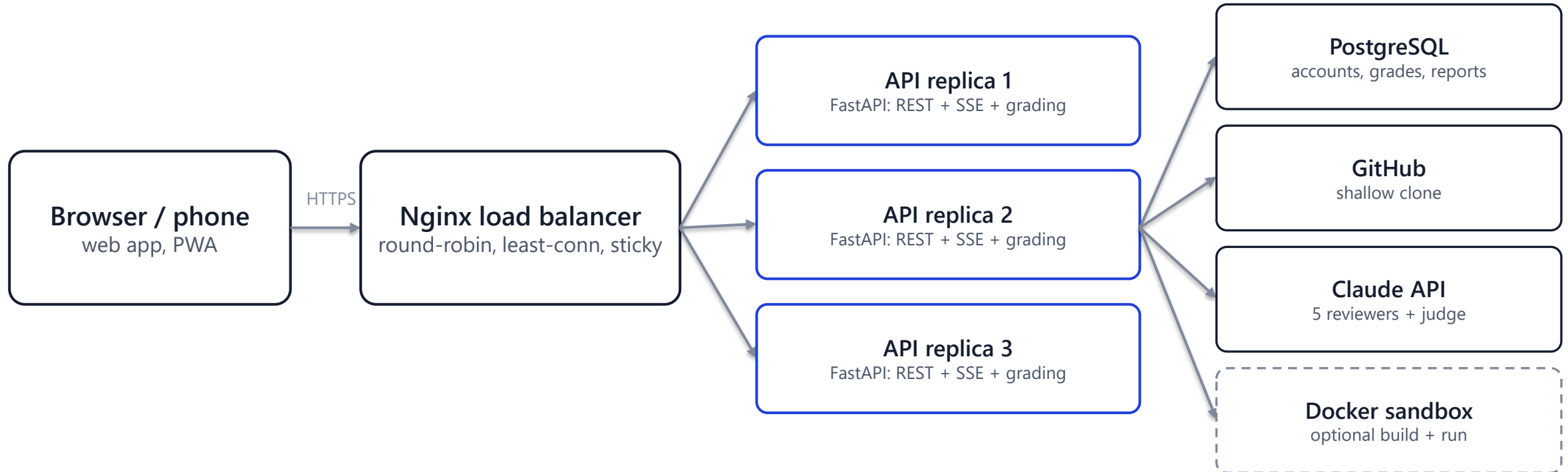
unit, integration, CI

Docker & DevOps

images, compose, delivery

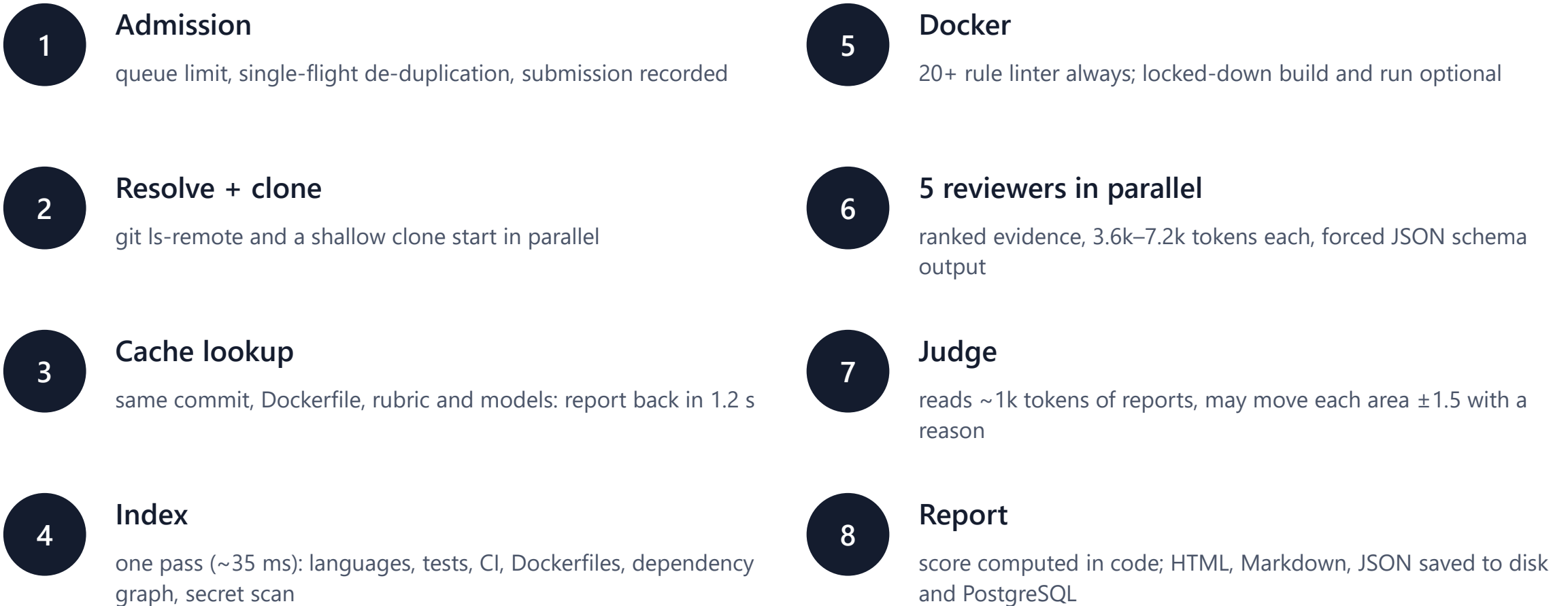
Runs on Claude models (LLM mode) or, with no API key, on rule-based scorers through the same pipeline (heuristic mode).

System architecture



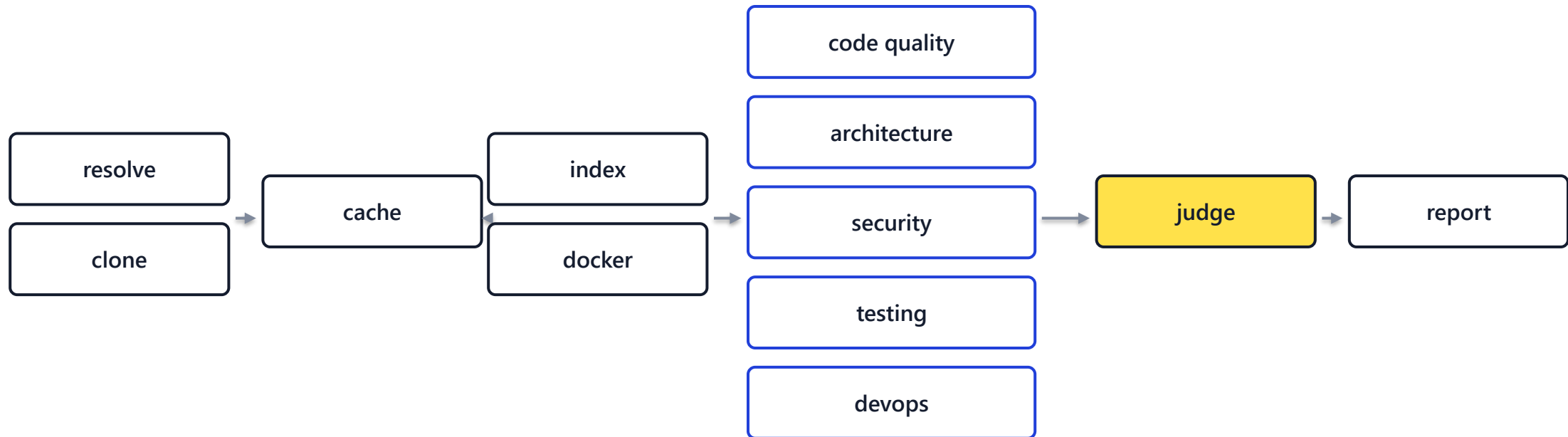
- **Stateless replicas:** all shared state (users, sessions, submissions, reports) lives in PostgreSQL, so any replica serves any page.
- **Durable event log:** every progress event is also stored in PostgreSQL, so any replica can stream any job (no sticky routing needed).
- **Deployed:** one container on Render + Neon PostgreSQL; locally docker compose runs Nginx + 3 replicas + PostgreSQL.

What happens to one submission



The browser follows every stage live over Server-Sent Events; if the stream drops it falls back to polling.

Stages overlap: the pipeline is a graph



$$T_{\text{total}} \approx \max(T_{\text{resolve}}, T_{\text{clone}}) + \max(T_{\text{index}} + \max(T_{\text{reviewer 1-4}}), T_{\text{docker}} + T_{\text{devops}}) + T_{\text{judge}}$$

Only the DevOps reviewer waits for Docker; the other four start right after indexing, so a slow build overlaps with model calls. Each report records its measured critical path and the speed-up over running the same stages in sequence.

Multi-agent design

Agent	Area	Evidence it sees (picked from the shared index)
Code Quality	readability, size, duplication	largest files, a diverse sample, duplication and comment metrics
Architecture	layers, coupling, API/DB design	entrypoints, routes/services/models, dependency graph and cycles, compose
Security	OWASP Top 10	secret-scanner and risky-pattern hits (redacted), auth files, .gitignore
Testing	unit, integration, e2e, CI	test files, test-to-source ratio, CI workflows, test scripts
DevOps & Docker	containers and delivery	Dockerfile, 20+ lint rules, build/run result, compose, CI
Judge	final verdict	only the five compact structured reports, never raw code

- **Structured output:** every call uses forced tool use with a JSON schema; scores are clamped and validated in code.
- **Bounded judge:** can move each area by at most ± 1.5 and must give a rationale; it calibrates, it does not re-grade.
- **Graceful degradation:** a reviewer that errors or misses its deadline falls back to its rule-based scorer, and the report says so.

Scoring is computed in code

```
final score = 10 ×  $\Sigma$  (weight_area × score_area)
```

- **Each area** is scored 0–10 by its reviewer, then calibrated by the judge (± 1.5 at most).
- **Weights** are set per assignment and shown to students before they submit (default: code 25%, architecture 25%, testing 20%, security 15%, DevOps 15%).
- **Best attempt counts:** students can resubmit as often as they want.
- **Instructor adjustment:** a score can be set by hand with a written reason; the original is kept and can be restored, and the student sees the reason.

Grade	A	A-	B	B-	C	C-	D	F
Score from	85	78	70	62	55	48	40	0

Why in code?

The same area scores always give the same total.

Every grade can be audited and explained.

Models judge quality; arithmetic stays deterministic.

Latency engineering

Technique	Effect
Fan-out of 5 reviewers	model phase costs the slowest reviewer, not the sum
Docker overlapped with reviewers	build leaves the critical path
Speculative clone ls-remote	a cache miss pays nothing for resolution
Content-addressed result cache	unchanged code re-graded in 1.2 s
Single-flight de-duplication	identical in-flight requests share one job
One-pass indexer	35 ms for an 8-service repository
Ranked evidence + char budgets	3.6k–7.2k input tokens per reviewer
Shared prompt prefix	byte-identical across reviewers for prompt caching
Capped output	≤ 7 findings, short summaries (decoding dominates latency)
Non-blocking I/O	git, docker, indexing in worker threads; SSE keeps flowing

Measured results



Benchmark: scripts/latency_benchmark.py on dockersamples/example-voting-app, real clone, index, lint and judge logic; model latency simulated (0.8 s to first token, 20k in / 70 out tokens per second).

Stage	Typical	Hard limit
Resolve + clone	0.8–2.5 s	120 s
Index	< 0.1 s	5,000 files, 40 MB
Each reviewer	10–25 s	120 s, then rule-based fallback
Docker build + run	20–300 s	600 s + 8 s run
Judge	8–15 s	150 s, then rule-based

Observed, not guessed

GET /api/metrics returns rolling p50 / p95 / max per stage.
Platform health and How it works show them live.
Critical path: clone → index → security → judge → report.

Reliability and scaling

- **Timeouts everywhere:** git, Docker, every model call and the judge.
- **Retries with backoff** for 429 and 5xx; a global semaphore queues model calls instead of causing retry storms.
- **Admission control:** a concurrency limit with a visible queue position.
- **Heartbeats every 20 s:** a replica that stops is detected; its unfinished gradings are failed and its files cleaned.
- **Reports survive restarts:** atomic writes plus a copy in PostgreSQL (free hosts wipe the disk).
- **Replayable streams:** events are stored in order; any replica serves them, and a dead replica's job ends with its saved outcome.

Scaling path

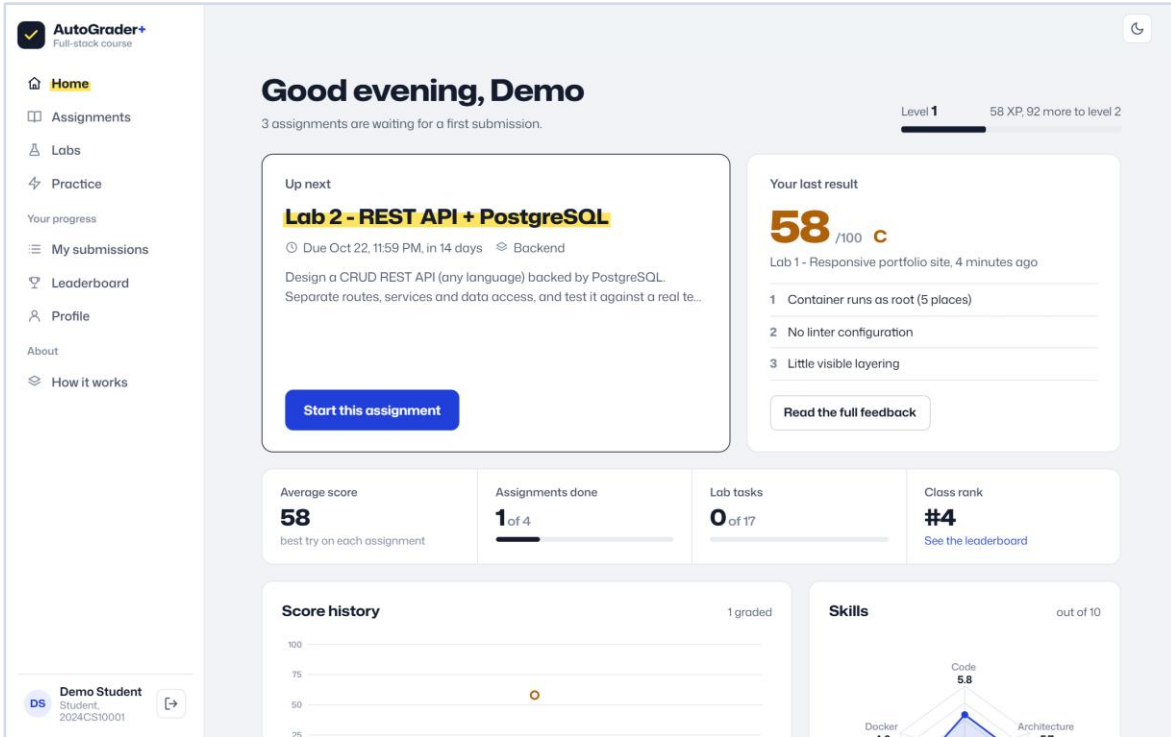
Today: stateless replicas + durable event log: add replicas to scale.

1. PostgreSQL work queue: workers claim jobs with FOR UPDATE SKIP LOCKED; jobs survive restarts.
2. Separate web and worker roles (Autolab-style queue + worker pool).
3. Isolated Docker build workers, a fresh environment per submission.
4. Reports in object storage; retry one reviewer, not the whole job.
5. Courses: one deployment, many classes.

Security

Threat	Defence
Malicious repository input	only https://github.com/owner/repo; validated refs; no shell; git never prompts
Secrets in student code	9 scanner rules; values redacted before any prompt or report
Untrusted Docker builds	no network, 512 MB, 1 CPU, 256 pids, all caps dropped; opt-in only
Stolen database	bcrypt password hashes; session tokens stored only as SHA-256
Students reading others' data	every instructor route checks the role on the server; reports only for their owner
Password guessing	throttled per client IP (Cloudflare header, never forgeable X-Forwarded-For) and per account
Public demo misuse	only a demo student login is public; sign-up closed; demo logins read-only
Grade tampering	adjusted grade pinned in every view until restored; no student endpoint writes a score
Cross-site attacks	SameSite + Secure cookies, cross-site request blocking, strict CSP, HSTS, no framing
Spreadsheet injection	CSV gradebook neutralises formulas

Student view: what to do next



The dashboard shows a sidebar with navigation links: Home, Assignments, Labs, Practice, Your progress, My submissions, Leaderboard, Profile, About, and How it works. The main content area is titled "Good evening, Demo" and shows "Level 1" with 58 XP, 92 more to level 2. It features a "Up next" section for "Lab 2 - REST API + PostgreSQL" due on Oct 22, 11:59 PM, with a description and a "Start this assignment" button. A "Your last result" section shows a score of 58/100 (C) for "Lab 1 - Responsive portfolio site" with a list of three items to fix: "Container runs as root (5 places)", "No linter configuration", and "Little visible layering". Below this are four summary cards: "Average score 58", "Assignments done 1 of 4", "Lab tasks 0 of 17", and "Class rank #4". At the bottom, there's a "Score history" chart and a "Skills" radar chart.

AutoGrader+
Full-stack course

Home

Assignments

Labs

Practice

Your progress

My submissions

Leaderboard

Profile

About

How it works

Good evening, Demo

3 assignments are waiting for a first submission.

Level 1 58 XP, 92 more to level 2

Up next

Lab 2 - REST API + PostgreSQL

Due Oct 22, 11:59 PM, in 14 days Backend

Design a CRUD REST API (any language) backed by PostgreSQL. Separate routes, services and data access, and test it against a real te...

Start this assignment

Your last result

58 /100 C

Lab 1 - Responsive portfolio site, 4 minutes ago

- 1 Container runs as root (5 places)
- 2 No linter configuration
- 3 Little visible layering

Read the full feedback

Average score **58**
best try on each assignment.

Assignments done **1** of 4

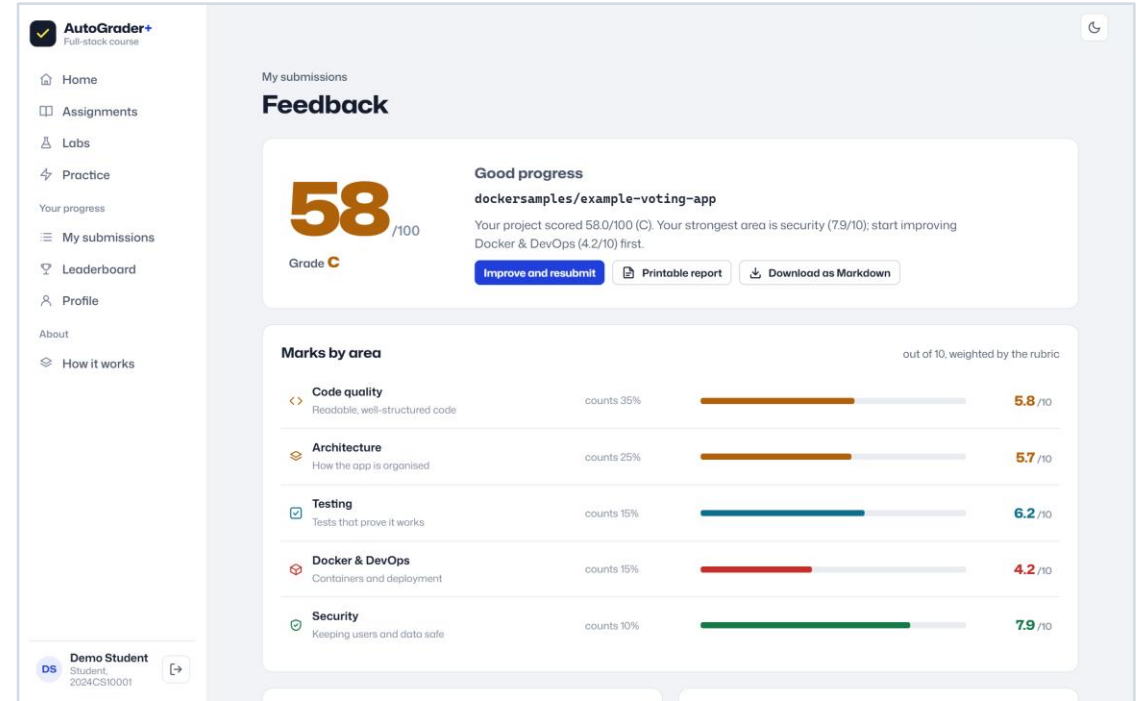
Lab tasks **0** of 17

Class rank **#4**
See the leaderboard

Score history 1 graded

Skills out of 10

Demo Student
Student, 2024CS10001



The feedback page shows a sidebar with navigation links: Home, Assignments, Labs, Practice, Your progress, My submissions, Leaderboard, Profile, About, and How it works. The main content area is titled "My submissions Feedback" and shows a score of 58/100 (C) for "dockersamples/example-voting-app". It includes a "Good progress" message and a list of marks by area with progress bars and scores. At the bottom, there's a "Demo Student" card.

AutoGrader+
Full-stack course

Home

Assignments

Labs

Practice

Your progress

My submissions

Leaderboard

Profile

About

How it works

My submissions Feedback

58 /100 Grade C

Good progress
dockersamples/example-voting-app

Your project scored 58.0/100 (C). Your strongest area is security (7.9/10); start improving Docker & DevOps (4.2/10) first.

Improve and resubmit Printable report Download as Markdown

Marks by area

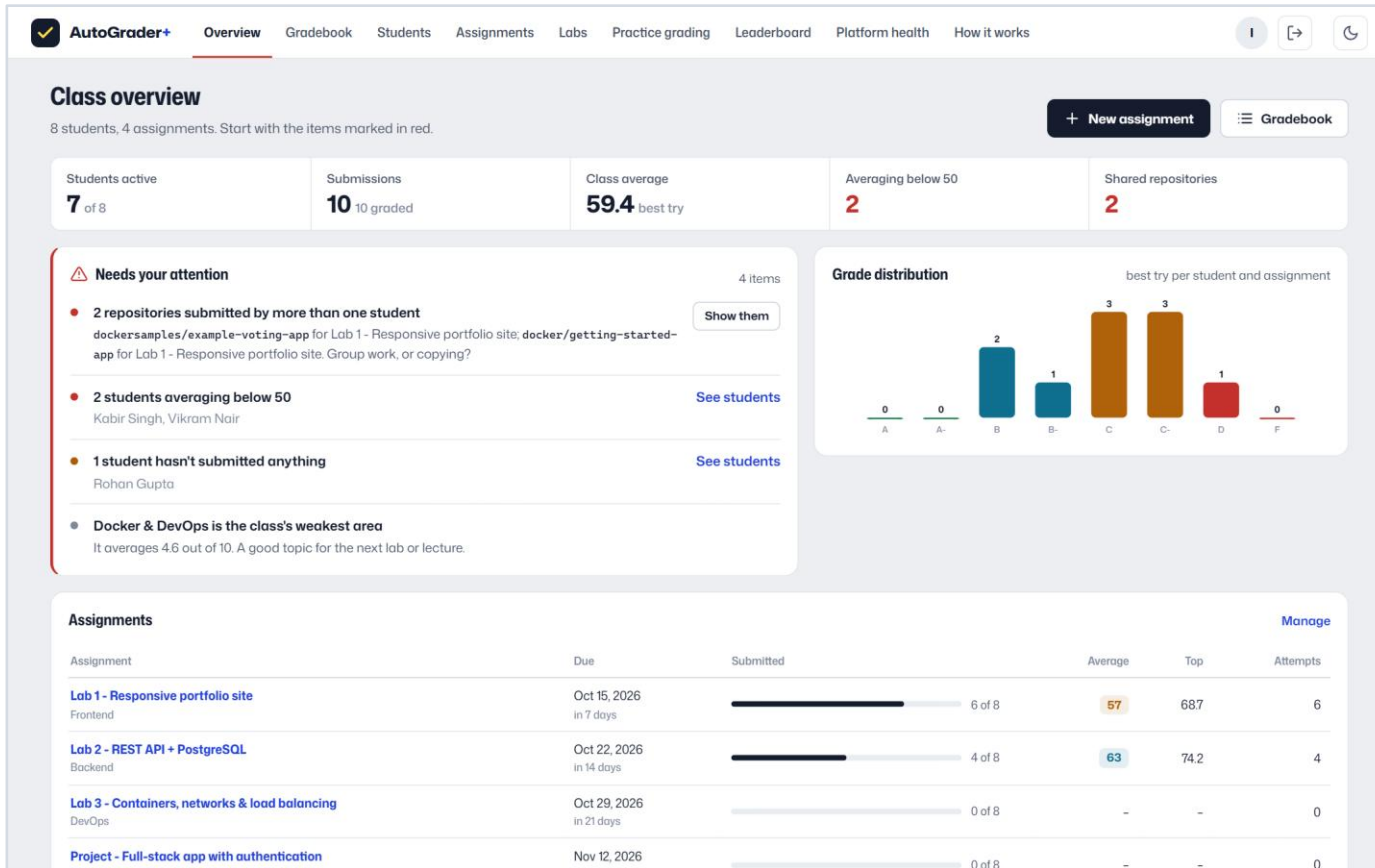
out of 10, weighted by the rubric

Code quality	Readable, well-structured code	counts 35%	5.8 /10
Architecture	How the app is organised	counts 25%	5.7 /10
Testing	Tests that prove it works	counts 15%	6.2 /10
Docker & DevOps	Containers and deployment	counts 15%	4.2 /10
Security	Keeping users and data safe	counts 10%	7.9 /10

Demo Student
Student, 2024CS10001

- **Dashboard:** the next assignment, the last result with the first three things to fix, progress, skills, deadlines.
- **Feedback:** the score, marks by area with weights, numbered priorities with file names, topics to learn, every finding with its fix.

Instructor view: what needs attention



- **Needs your attention:** shared repositories, failed gradings, students below 50, students who haven't started, low completion near a deadline, the weakest area.
- **Class numbers:** active students, submissions, class average.
- **Assignments:** completion, average and top score per assignment.
- **Latest submissions:** filter by shared repo, failed, in progress.

Gradebook and assignment analytics

AutoGrader+ Overview **Gradebook** Students Assignments Labs Practice grading Leaderboard Platform health How it works

Overview

Gradebook

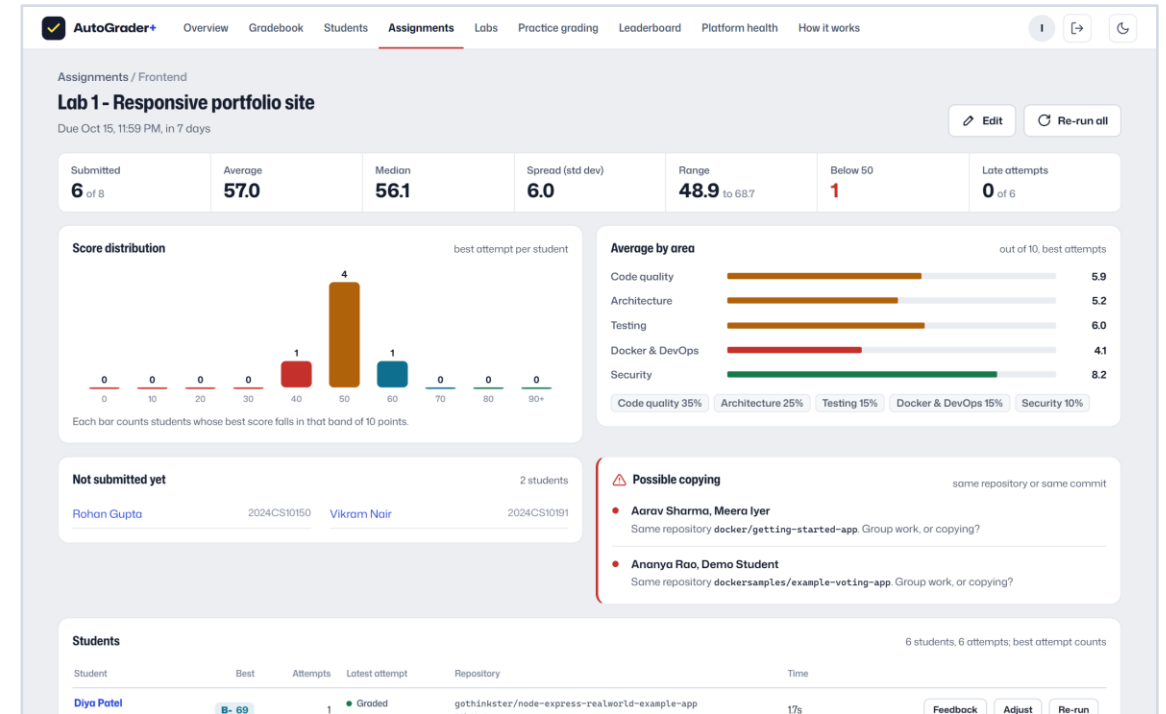
Best graded attempt for every student and assignment. Open a score for the feedback, a name for the student, a column title for that assignment's analytics.

[Export CSV](#)

Search students

A B C D/F • late adj adjusted Missing past deadline

Student	Lab 1 - Responsive portfolio site due Oct 15, 2025	Lab 2 - REST API + PostgreSQL due Oct 22, 2025	Lab 3 - Containers, networks & load balancing due Oct 29, 2025	Project - Full-stack app with authentication due Nov 12, 2025	Average
Aarav Sharma 2024CS10011	54 C-	60 C	-	-	57
Ananya Rao 2024CS10168	58 C	71 B	-	-	64
Demo Student 2024CS10001	58 C	-	-	-	58
Diya Patel 2024CS10042	69 B-	74 B	-	-	72
Kabir Singh 2024CS10077	49 C-	-	-	-	49
Meera Iyer 2024CS10103	54 C-	-	-	-	54
Rohan Gupta 2024CS10150	-	-	-	-	-
Vikram Nair 2024CS10191	-	46 D	-	-	46
Class average	57.0 6 graded	62.9 4 graded	-	-	



- **Gradebook:** every student × assignment, late and adjusted marks, missing work after the deadline, class averages, CSV export.
- **Analytics:** mean, median, spread, range, distribution, area averages, submissions per day, who hasn't submitted, possible copying.
- **Actions:** adjust a grade with a reason, re-run one submission, or re-run the whole assignment after changing its rubric.

Hands-on labs

The screenshot shows the AutoGrader+ interface for a database lab. On the left is a sidebar with navigation links: Home, Assignments, Labs (highlighted), Practice, Your progress, My submissions, Leaderboard, Profile, About, and How it works. The main content area is titled 'Databases: SQL & indexes' and includes a description: 'Query a real relational schema (students, courses, enrollments) and read query plans.' Below this is a 'Query editor' section with a 'Task to check (optional)' dropdown set to 'Free play: no check'. The query editor contains the text 'SELECT * FROM courses;'. Below the editor are buttons for 'Run query', 'Index on enrollments(student_id)', and 'Show a solution'. To the right of the query editor is a 'Tables' section displaying the schema for the database, including tables for students, courses, and enrollments. At the bottom right is a 'Your tasks' section with two tasks: 'Filter and sort: CSE students in year 3, by name' and 'Join three tables: who is enrolled in CSL100?'. The bottom left of the interface shows the user's identity as 'Demo Student' with ID '2024CS10001'.

AutoGrader+ Full-stack course

Home Assignments Labs Practice Your progress My submissions Leaderboard Profile About How it works

Labs / Database

Databases: SQL & indexes

Query a real relational schema (students, courses, enrollments) and read query plans.

Query editor Ctrl + Enter to run

Task to check (optional)

Free play: no check

```
SELECT * FROM courses;
```

Run query Index on enrollments(student_id) Show a solution

Tables

```
CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT NOT NULL, branch TEXT NOT NULL, year INTEGER NOT NULL);
CREATE TABLE courses (id INTEGER PRIMARY KEY, code TEXT NOT NULL UNIQUE, title TEXT NOT NULL, credits INTEGER NOT NULL);
CREATE TABLE enrollments (student_id INTEGER NOT NULL REFERENCES students(id), course_id INTEGER NOT NULL REFERENCES courses(id), semester TEXT NOT NULL, grade_points INTEGER, PRIMARY KEY (course_id, student_id));
```

240 students · 10 courses · ~1,100 enrollments. Every query runs on a fresh, read-only, in-memory copy, with a 0.5 s limit.

Your tasks 0 of 4 done

- ☐ Filter and sort: CSE students in year 3, by name +20 XP
- ☐ Join three tables: who is enrolled in CSL100? +20 XP

DS Demo Student Student, 2024CS10001

- **Frontend:** HTML, CSS and JS editor with a sandboxed live preview and DOM checks.
- **Databases:** SQL on a real schema; the server checks answers; see an index change the query plan.
- **Load balancers:** real requests through Nginx to the replicas: round-robin, least-connections, sticky.
- **Networks:** round-trip time, jitter, proxy headers, Server-Timing breakdown.
- **Docker:** fix a Dockerfile with the same 20+ rule linter the grader uses.
- **XP only for server-verified tasks**, so the leaderboard can't be inflated from the browser.

Data model, stack and deployment

Table	Stores
users	students and instructors, script hash, role
sessions	SHA-256 of session tokens, expiry
assignments	brief, track, rubric weights, checklist, deadline
submissions	each attempt: repo, status, score, area scores, priorities
grade_overrides	instructor adjustments: original, new, reason, who
job_events	progress events, so any replica can stream any job
report_artifacts	full HTML / Markdown / JSON reports
lab_progress	completed lab tasks

Tech stack

Python 3.12, FastAPI, asyncio, Pydantic v2
Claude API (forced tool use) or heuristics
PostgreSQL (psycopg 3) or SQLite
HTML/CSS/JS modules, SVG charts, SSE, PWA

Deployment

Docker: multi-stage, non-root, healthcheck
Render (free web service, auto-deploy from main)
Neon PostgreSQL (free, Singapore)
Local: docker compose = Nginx + 3 replicas + Postgres

Limitations and next steps

Limitations today

- A grading runs inside the server that accepted it; a restart fails it (the student resubmits).
- Docker builds run student RUN steps, so the sandbox is opt-in and meant for a dedicated VM.
- Copy detection compares repositories and commits, not code similarity.
- The free host sleeps after 15 minutes; the first visit takes about a minute.

Next steps

- PostgreSQL work queue (SKIP LOCKED) and worker roles: jobs survive restarts.
- Token-level similarity (JPlag-style) across submissions.
- Per-student containers so each student's backend runs on the platform.
- Regrade requests from students, deadline extensions per student.

Thank you

Live demo

- 1 Open autograder-plus.onrender.com (allow a minute if it was asleep)
- 2 Sign in → Student view: dashboard, feedback, labs
- 3 Submit a repository and watch the reviewers run
- 4 Sign in with the instructor account
- 5 Overview → Gradebook → an assignment's analytics
- 6 How it works: architecture and live latency

Links

Live: autograder-plus.onrender.com

Code: github.com/roshanraj9136/auto-grader

Design: [docs/ARCHITECTURE.md](#)

Report: [docs/AutoGrader-Report.pdf](#)

Demo logins are on the sign-in page

Questions?